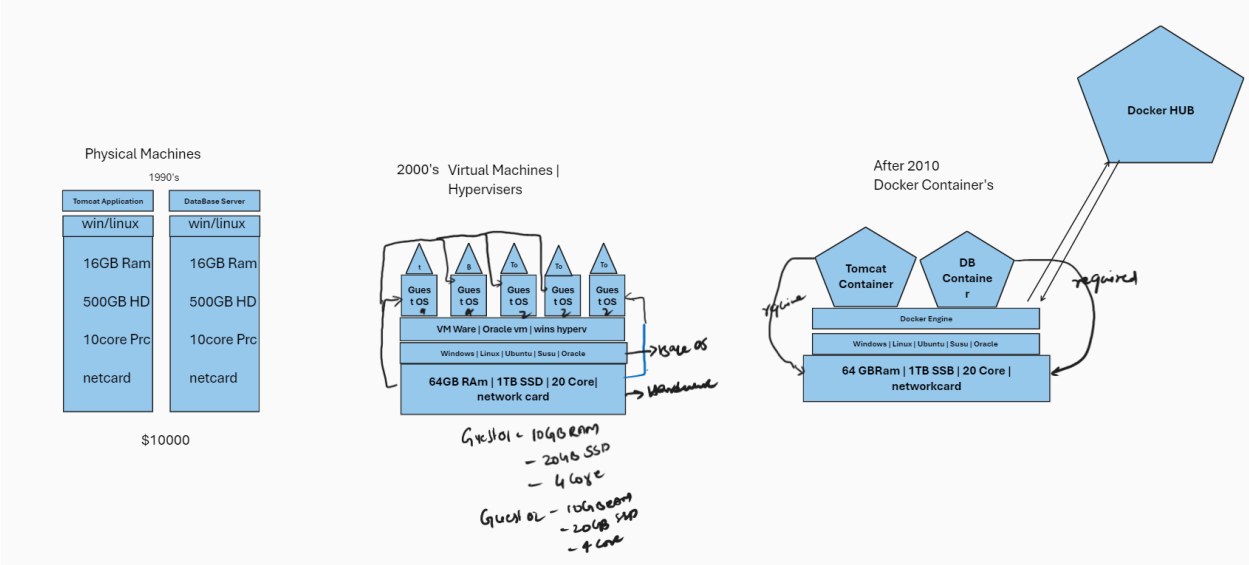
Docker

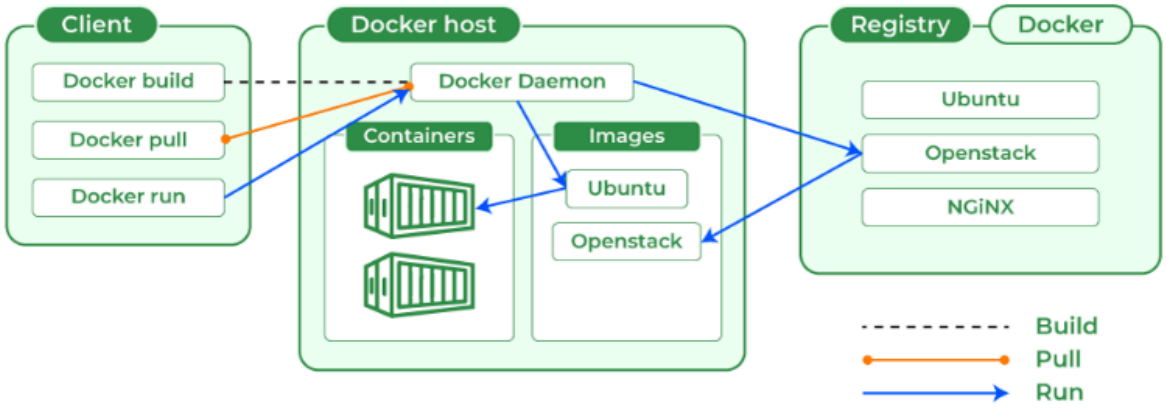
What is Docker



Docker is a powerful tool used for developing, packaging, and deploying applications efficiently. Docker is a container management service. Docker was released in 2013. It is open-source and available for different platforms like Windows, macOS, and Linux.

Docker makes use of a client-server architecture. The Docker client talks with the docker daemon which helps in building, running, and distributing the docker containers. The Docker client runs with the daemon on the same system or we can connect the Docker client with the Docker daemon remotely. With the help of REST API over a UNIX socket or a network, the docker client and daemon interact with each other.

Docker



Docker is a platform that simplifies the process of developing, deploying, and running applications by using containerization technology. Here's a basic rundown of its key features:

1. **Containers**: Docker packages applications and their dependencies into containers. Containers are lightweight, portable, and can run consistently across different computing environments, such as a developer's laptop, a staging environment, or a production server.
2. **Images**: A Docker image is a snapshot of a container that includes everything needed to run an application, such as the code, runtime, libraries, and environment variables. Images are built from a Dockerfile, which specifies the setup and configuration of the application.
3. **Docker Engine**: This is the runtime that executes containers. It manages the containers, ensuring they are running as expected.
4. **Docker Hub:** This is a cloud-based registry where Docker images can be stored and shared. Users can pull pre-built images from Docker Hub or push their own images for others to use.
5. **Isolation and Consistency:** Docker containers provide process and filesystem isolation, which means applications run in their own environment without interfering with other applications. This isolation

ensures that applications behave the same way regardless of where they are deployed.

6. **Portability:** Since containers encapsulate the application and its dependencies, you can move them across different environments without worrying about compatibility issues. This makes it easier to deploy applications on various platforms and cloud providers.

Docker Commands

Working on Images

---

1. To download a docker image

   docker pull image_name
2. To see the list of docker images

   docker image ls

   (or)

   docker images
3. To delete a docker image from docker host

   docker rmi image_name/image_id
4. To upload a docker image into docker hub

   docker push image_name
5. To tag an image

   docker tag image_name ipaddress_of_local_registry:5000/image_name
6. To build an image from a customized container

Docker

docker commit container_name/container_id new_image_name

7. To create an image from docker file

   docker build -t new_image_name

8. To search for a docker image

   docker search image_name

9. To delete all images that are not attached to containers

   docker system prune -a

Working on containers

10. To see the list of all running containers

    docker container ls

11. To see the list of running and stopped containers

    docker ps -a

12. To start a container

    docker start container_name/container_id

13. To stop a running container

    docker stop container_name/container_id

14. To restart a running container

    docker restart container_name/container_id

    To restart after 10 seconds

    docker restart -t 10 container_name/container_id

15. To delete a stopped container

    docker rm container_name/container_id

Docker

16. To delete a running container

    docker rm -f container_name/container id

17. To stop all running containers

    docker stop $(docker ps -aq)

18. To restart all containers

    docker restart $(docker ps -aq)

19. To remove all stopped containers

    docker rm $(docker ps -aq)

20. To remove all containers(running and stopped)

    docker rm -f $(docker ps -aq)

21. To see the logs generated by a container

    docker logs container_name/container_id

22. To see the ports used by a container

    docker port container_name/container_id

23. To get detailed info about a container

    docker inspect container_name/container_id

24. To go into the shell of a running container which is moved into background

    docker attach container_name/container id

25. To execute any command in a container

    docker exec -it container_name/container_id command

    Eg: To launch the bash shell in a container

Docker

docker exec -it container_name/container_id bash

26. To create a container from a docker image (imp)

    docker run image_name

Run command options

- -it
  (for opening an interactive terminal in a container)
- --name
  (Used for giving a name to a container)
- -d
  (Used for running the container in detached mode as a background process)
- -e
  (Used for passing environment variables to the container)
- -p
  (Used for port mapping between port of container with the docker host) port.
- -P
  (Used for automatic port mapping i.e., it will map the internal port of the container with some port on the host machine. This host port will be some number greater than 30000)
- -v
  (Used for attaching a volume to the container)
- --volume-from
  (Used for sharing volume between containers)
- --network
  (Used to run the container on a specific network)

Docker

- --link
  (Used for linking the container for creating a multi container architecture)
- --memory
  (Used to specify the maximum amount of ram that the container can use)

scenario 1: Tomcat

Scenario 2: Jenkins

Scenario 3: Nginx

Scenario 4:

- Run mysql container and pass the environment variable (password)
  docker run --name test-sql-server -d -e MYSQL_ROOT_PASSWORD=hint mysql

- docker exec -it test-sql-server /bin/bash
  mysql -u root -p → Enter password

- Create database

```
CREATE DATABASE HINTechnologies;

use HINTechnologies

    CREATE TABLE information_schemamysql(EmployeeID
INT PRIMARY KEY AUTO_INCREMENT,FirstName VARCHAR(50)
NOT NULL,LastName VARCHAR(50) NOT NULL,Email
VARCHAR(100),Phone VARCHAR(20),HireDate DATE,Salary
DECIMAL(10,2),Department VARCHAR(50));
```

Multi container architecture using docker

In Docker, the term "multi-container" generally refers to the use of multiple Docker containers working together to run a complex application or service. Each container typically runs a specific part of the application or service, and they interact with each other to function as a cohesive system.

1. - - link
2. docker compose

⇒ -- link

The --link option was a feature in Docker used to create a connection between containers, allowing them to communicate with each other. It provided a way to link one container to another by establishing environment variables and network connections. However, this feature is now deprecated and has been largely replaced by Docker's networking features and Docker Compose.

Use Case 01 (Create 2 containers and link between them)

Docker

1. Create first centos container

   docker run --name dev01 -it centos

   (to come out from the prompt without exit **ctrl+p+q**)

2. Create 2nd centos container establish a link with container1

   docker run --name dev02 --link dev01:dev01-container -it centos

   Note: dev01-container is an Alias Name

3. To check the link is established

   ping from dev02 to dev01

   → To get the ip Address assigned to these containers , use docker inspect command.

   docker inspect dev02 (or) docker inspect dev01

Use Case 02 (Create development environment using docker)

→ MySql Container

→ wordpress container

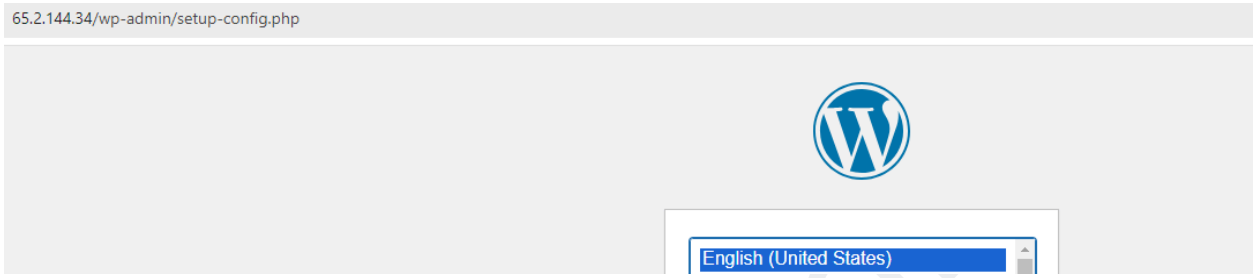
1. For this use case we are creating frontend UI and backend Database , here front end is wordpress and backend is Database
2. Create 2 containers

   docker run --name dev-db -d -e MYSQL_ROOT_PASSWORD=hint mysql

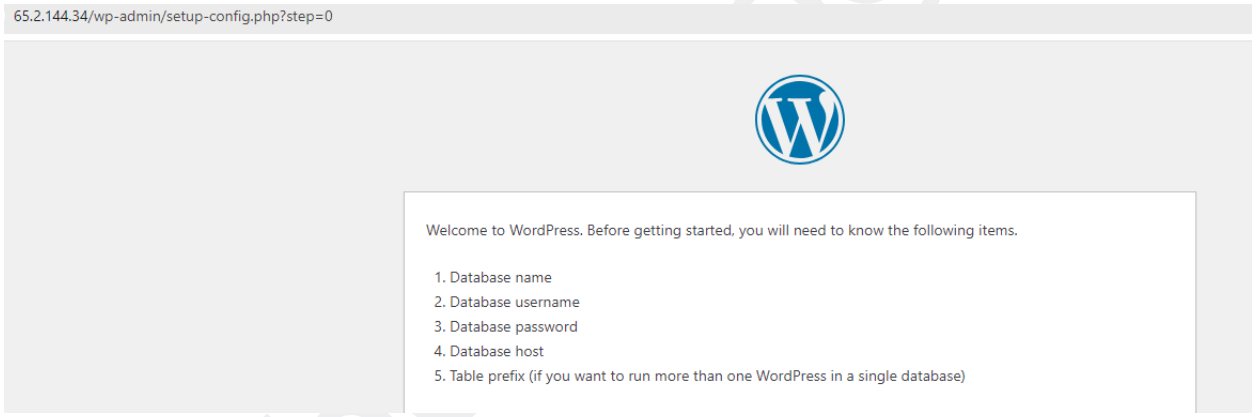
Docker

docker run --name dev-UI -d -p 80:80 --link dev-db:mysql wordpress

3. Now open wordpress with the public ip+post of the EC2 Instance
4. You can see the login page of the wordpress



Next



Use Case 03 (LAMP Architecture Using docker)

LAMP is a popular acronym representing a stack of open-source software used for web development and hosting. It stands for:

Linux: The operating system.
Apache: The web server.
MySQL: The database management system (though MariaDB is often used as a drop-in replacement)
PHP: The programming language (or sometimes Python or Perl, though PHP is the most common).

Docker

1. Create the below containers

   MySql Container:

   docker run --name mysqldb -d -e MYSQL_ROOT_PASSWORD=hint mysql

   Tomcat Container

   docker run --name webserver -d -p 8080:8080 --link mysqldb:mysql tomee

   PHP Container

   docker run --name php -d --link apache:tomcat --link mysqldb:mysql php

2. Check the containers are linked with mysqldb

Docker compose

Docker Compose is a tool designed to simplify the management of multi-container Docker applications. It allows you to define and run multi-container Docker applications using a single YAML file, making it easier to manage complex setups involving multiple services.

Basic Workflow

1. **Define the Application**: Create a docker-compose.yml file to define the services and their configurations.
2. **Start the Application**: Use the docker-compose up command to start all the services defined in the docker-compose.yml file. Docker Compose will handle building images, creating containers, and setting up networks and volumes as needed.

3. **Stop and Remove the Application**: Use docker-compose down to stop and remove all containers, networks, and volumes created by docker-compose up.
4. **View Logs**: Use docker-compose logs to view the logs of all running services.
5. **Scale Services**: Use docker-compose up --scale service_name=num to scale a particular service.

Install docker compose :

1. Check Docker Compose Installation

   docker-compose --version
2. Download the latest version docker compose

   sudo curl -L "https://github.com/docker/compose/releases/download/$(curl -s https://api.github.com/repos/docker/compose/releases/latest | grep 'tag_name' | cut -d\" -f4)/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose

3. Give the executable permissions

   chmod +x /usr/local/bin/docker-compose

4. Now verify the installation

   docker-compose --version

```
---
version: "1"
services:
  web:
    image: nginx:latest
```

```
    ports:
      - 80:80
    depends_on:
      - db
  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: hint
      MYSQL_DATABASE: mydb
    volumes:
      - db-data:/var/lib/mysql
volumes:
  db-data: null
```

5. To run the docker-compose.ylm file

   docker-compose up

Docker Volumes

Docker volumes are a mechanism for persisting data generated and used by Docker containers. They provide a way to store data outside the container's filesystem, which is crucial for data persistence and sharing between containers. Here's an overview of Docker volumes:

Key Characteristics of Docker Volumes:

1. **Persistent Storage:**

   Data stored in a Docker volume persists even if the container is removed. This is in contrast to the data stored in the container's filesystem, which is ephemeral and lost when the container is deleted.

2. **Separate from Container Filesystem:**

   Volumes are managed by Docker and are stored in a part of the host filesystem which is managed by Docker itself, typically under /var/lib/docker/volumes/. This separation means that volumes are not tied to the lifecycle of a single container.

3. **Shared Across Containers:**

   Volumes can be shared and reused between multiple containers. This is useful for scenarios where multiple containers need access to the same data.

4. **Backup and Restore:**

   Data in volumes can be backed up or migrated easily. You can use Docker commands to export the data or copy it from the volume to another location.

5. **Driver Support:**

   Docker supports different volume drivers that allow you to use various storage backends. For example, you can use local storage, networked storage, or even cloud storage solutions.

Basic Docker Volume Commands:

1. Create a Volume: This creates a new volume named my_volume.

   docker volume create my_volume

   Note : Above command create volume inside the docker volume create my_volume

Docker

2. List Volumes:This lists all Docker volumes on your system.

   docker volume ls

3. Inspect a Volume:This provides detailed information about a specific volume, such as its mount point and configuration.

   docker volume inspect my_volume

4. Remove a Volume:This removes the specified volume. Note that a volume cannot be removed if it is in use by a container.

   docker volume rm my_volume

5. Use a Volume in a Container: This command mounts the volume my_volume to /path/in/container inside the container my_container.

   docker run -d --name my_container -v my_volume:/path/in/container my_image

   ex: docker run -d --name test-container -v my_volume:/opt/ centos

Create physical folder and attach it to docker container

1. Login to the host machine and create a physical folder
   mkdir -p /opt/physical-volume

2. Add few files for the test purpose

```
touch /opt/physical-volume/file{1..3}.txt
```

3. Now attach this folder as volume

```
docker run -d -it --name test-container2 -v
/opt/physical-volume:/opt/physical-volume centos
```

To stop containers and remove

1. To stop docker container

```
docker stop container-name
```

2. To stop all containers

```
docker stop $(docker ps -q)
```

3. To remove the container

```
docker rm <container_id_or_name>
```

4. To remove all containers

```
docker rm $(docker ps -a -q)
```

5. To remove the Images

```
docker rmi <image_id_or_name>
```

6. To remove all the images

```
docker rmi $(docker images -q)
```

7. To stop the container and remove the image

```
docker stop $(docker ps -q) && \
docker rm $(docker ps -a -q) && \
docker rmi $(docker images -q)
```

Docker

Docker Swarm

Docker swarm is a container orchestration tool.

Docker Swarm is a native clustering and orchestration solution for Docker containers. It allows you to manage a group of Docker engines (nodes) as a single virtual Docker engine. Here are some key features and concepts:

1. **Cluster Management**: Docker Swarm turns a pool of Docker hosts into a single, virtual Docker host. You can deploy services across multiple nodes, and Swarm will handle the distribution and scaling of these services.
2. **Service Deployment**: You can deploy Docker containers as services, specifying how many replicas of each service you want to run. Swarm will ensure the desired number of replicas are running and will restart containers if they fail.
3. **Load Balancing**: Swarm provides built-in load balancing. When you deploy a service, Swarm automatically assigns an IP address and a DNS name to the service. Requests to this service are automatically load-balanced among the available replicas.

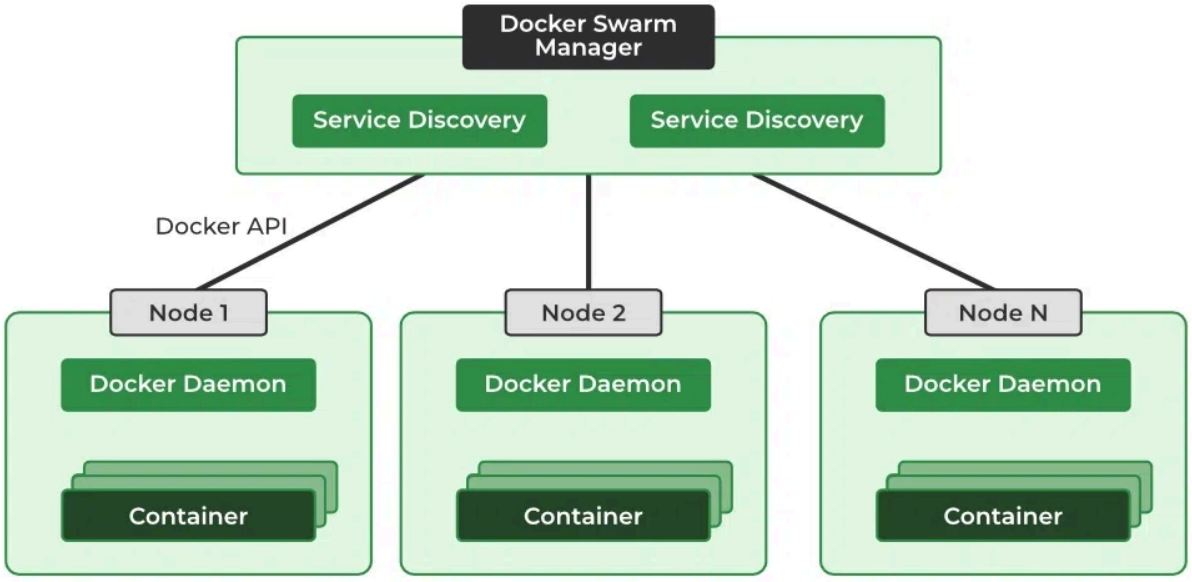
4. **Scaling**: You can easily scale services up or down with simple commands. Docker Swarm will handle the deployment and management of the scaled services.
5. **High Availability**: Swarm includes features for high availability, ensuring that services are distributed across nodes and that the cluster can handle node failures gracefully.
6. **Multi-Host Networking**: Swarm manages networking across different Docker hosts. It allows containers on different nodes to communicate with each other seamlessly.
7. **Security**: Docker Swarm provides security features like TLS encryption for communication between nodes and built-in role-based access control (RBAC).

Docker Swarm Architecture

There are two types of nodes in Docker Swarm:

1. **Manager node:** Carries out and oversees cluster-level duties.

2. **Worker node:** Receives and completes the tasks set by the manager node.



Different Modes of Docker Swarm

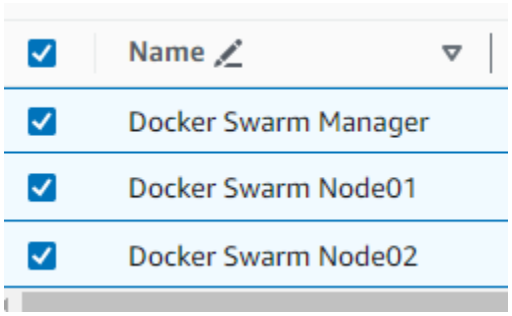
Docker Swarm mainly consists of two modes they are:

1. **Global Mode:** In this mode, Docker Swarm will maintain containers in all the slave nodes and master nodes. It will maintain the replicas of containers in all the nodes which are available in the cluster.
2. **Replicated Mode:** In this mode, Docker Swarm will deploy the containers based on the no.of replicas required for you. If you require 3 replicas it will deploy the containers based on the node availability.

Docker

Use Case 01 (Setup docker swarm on AWS)

1. Create 3 EC-2 Instances and give the below user data . And name it as
   Docker Swarm Manager
   Docker Swarm Node01
   Docker Swarm Node02




```
#!/bin/bash
# Update the package index
sudo yum update -y
# Install Docker from the Amazon Linux 2 repository
sudo yum install -y docker
# Start the Docker service
sudo systemctl start docker
# Enable Docker to start on boot
sudo systemctl enable docker
# Add the ec2-user to the Docker group
sudo usermod -aG docker ec2-user
# Verify Docker installation
docker --version
```

2. Connecte the First Instance (Docker Swarm Manager) and execute the below commands
   - → docker swarm init

Docker

```
[ec2-user@ip-172-31-42-233 ~]$ docker --version
Docker version 25.0.3, build 4debf41
[ec2-user@ip-172-31-42-233 ~]$ docker swarm init
Swarm initialized: current node (w0mrolrbzh1oo5l3vo7gleta6) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-1it0p8q8hwuqzfz2ily7pnvn1rkjg96nqq1zsp9p42lm5jkg7q-2t7x1wug23ftnf5gwj3bltbfv 172.31.42.233:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

[ec2-user@ip-172-31-42-233 ~]$
```

→ Swarm initialized: current node (w0mrolrbzh1oo5l3vo7gleta6) is now a manager.

→ To add a worker to this swarm, run the following command:

→ docker swarm join --token SWMTKN-1-1it0p8q8hwuqzfz2ily7pnvn1rkjg96nqq1zsp9p42lm5jkg7q-2t7x1wug23ftnf5gwj3bltbfv 172.31.42.233:2377

→ To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

3. Connect to the Second Node (Docker swarm Node01) and execute the docker join

```
[ec2-user@ip-172-31-36-248 ~]$ docker swarm join --token SWMTKN-1-1it0p8q8hwuqzfz2ily7pnvn1rkjg96nqq1zsp9p42lm5jkg7q-2t7x1wug23ftnf5gwj3bltbfv 172.31.42.233:2377
This node joined a swarm as a worker.
[ec2-user@ip-172-31-36-248 ~]$
```

4. Connect to the 3rd Instance (Docker swarm Node02) and execute the docker join

```
[ec2-user@ip-172-31-39-13 ~]$ docker swarm join --token SWMTKN-1-1it0p8q8hwuqzfz2ily7pnvn1rkjg96nqq1zsp9p42lm5jkg7q-2t7x1wug23ftnf5gwj3bltbfv 172.31.42.233:2377
This node joined a swarm as a worker.
[ec2-user@ip-172-31-39-13 ~]$
```

5. Now Deploy Application in docker swarm cluster

→ docker service create --name webapp -p 80:8080 tomee

The above command is going to create a webapp application .

```
[ec2-user@ip-172-31-42-233 ~]$ docker service create --name webapp -p 80:8080  tomee
nwc4bqk8j11xfqnz3kgifpbx3
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service converged
[ec2-user@ip-172-31-42-233 ~]$ docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED              STATUS              PORTS      NAMES
bda94fb3d947   tomee:latest    "/bin/bash /  cacert…"   About a minute ago   Up About a minute   8080/tcp   webapp.1.vkuu12d5e2p37m43oupyw18xi
```

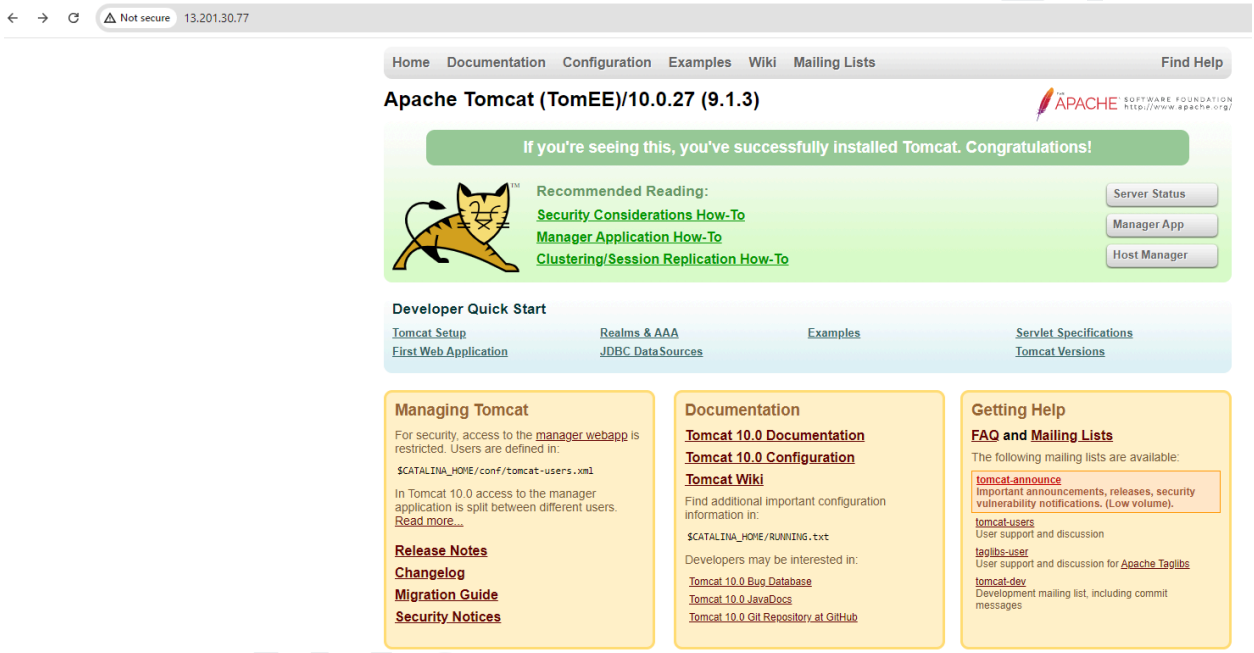
Docker

6. Create an replica to this application (Scale the created service from default 1 to 5 with the following command)

   → docker service scale webapp=5

   → docker ps (go and check in all node , how many containers created)

7. Now access the Application with the public ip of any of the node



8. We can add local DNS and access this application .

Docker swarm benefits

1. Simplified Setup and Management

- **User-Friendly**: Docker Swarm integrates seamlessly with Docker, making it easy to set up and manage clusters using familiar Docker CLI commands.
- **Quick Configuration**: Initializing and managing a Swarm cluster requires minimal configuration, streamlining the deployment process.

2. Scalability

- **Effortless Scaling**: Easily scale services up or down using simple Docker commands. Swarm handles the distribution of containers across the cluster based on your specifications.
- **Dynamic Resource Adjustment**: Automatically adjust the number of service replicas to meet fluctuating demand, optimizing resource use and performance.

3. High Availability

- **Fault Tolerance**: Services are replicated across multiple nodes to ensure availability. Swarm automatically redistributes tasks if a node fails, maintaining service continuity.
- **Resilient Infrastructure**: Provides built-in mechanisms to handle node failures and ensure that services remain operational even when issues occur.

4. Integrated Load Balancing

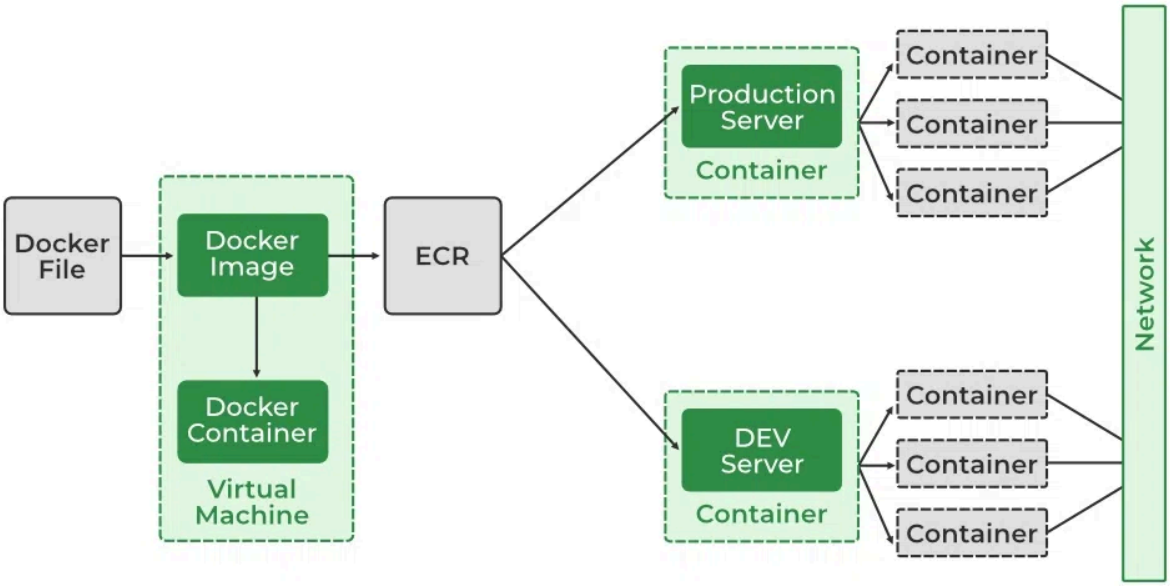
- **Automatic Traffic Distribution**: Docker Swarm includes built-in load balancing to distribute incoming network traffic across multiple containers, enhancing performance and preventing any single container from becoming a bottleneck.
- **Optimal Resource Utilization**: Ensures efficient use of resources by balancing loads, which helps to maximize the effectiveness of your containerized applications.

Docker Networking

Docker

Docker Networking refers to the set of mechanisms and technologies Docker provides for communication between Docker containers, as well as between containers and the outside world.

A network is a group of two or more devices that can communicate with each other either physically or virtually. The Docker network is a virtual network created by Docker to enable communication between Docker containers. If two containers are running on the same host they can communicate with each other without the need for ports to be exposed to the host machine.



Network Drivers

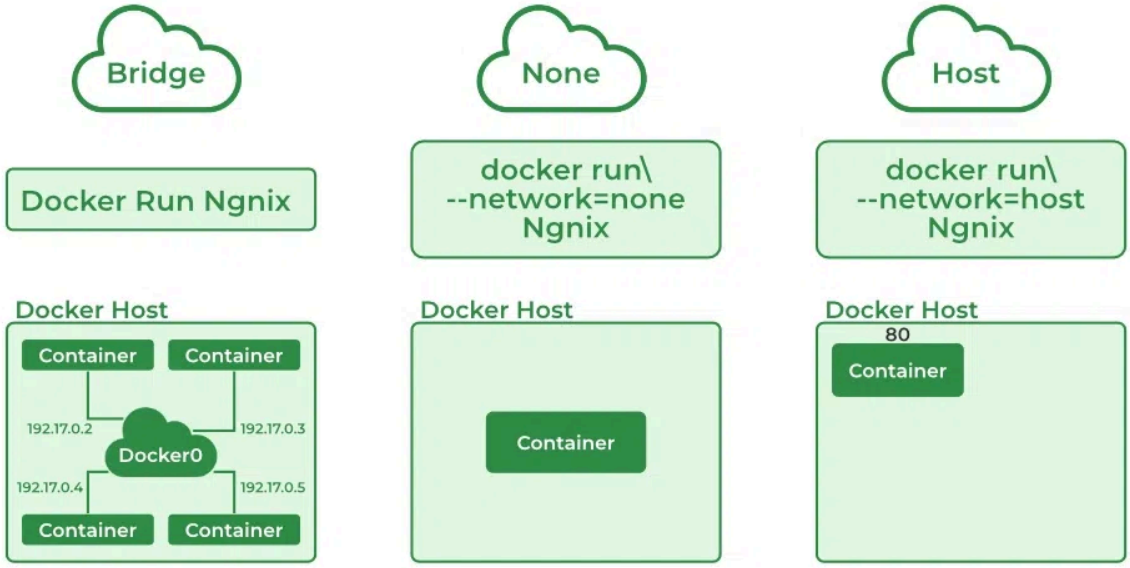
`docker network ls`

Types of Network Drivers

Docker

1. **bridge:** If you build a container without specifying the kind of driver, the container will only be created in the bridge network, which is the default network.
2. **host:** Containers will not have any IP address; they will be directly created in the system network which will remove isolation between the docker host and containers.
3. **none:** IP addresses won't be assigned to containers. These containers are not accessible to us from the outside or from any other container.
4. **overlay:** overlay network will enable the connection between multiple Docker demons and make different Docker swarm services communicate with each other.
5. **ipvlan:** Users have complete control over both IPv4 and IPv6 addressing by using the IPvlan driver.
6. **macvlan:** macvlan driver makes it possible to assign MAC addresses to a container.

Docker



Launch a Container on the Default Network

Use Case 01-

1. To create a docker network

docker network create --driver <driver-name> <bridge-name>

bridge dev

docker network create --driver bridge dev

2. To check the list of networks in docker host

Docker network ls

Docker

```
[root@ip-172-31-9-162 ~]# docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
9d6839ad08db   bridge    bridge    local
6ce2cd8dfb8b   dev       bridge    local
6182e6bfcb7e   host      host      local
3cd7e9df592f   none      null      local
[root@ip-172-31-9-162 ~]#
```

3. Attach new network to our docker container

   docker network connect dev e3bd635db546

   dev- Network Name

   E3bd635db546 - Container Name (or) Container ID

4. Inspect the network

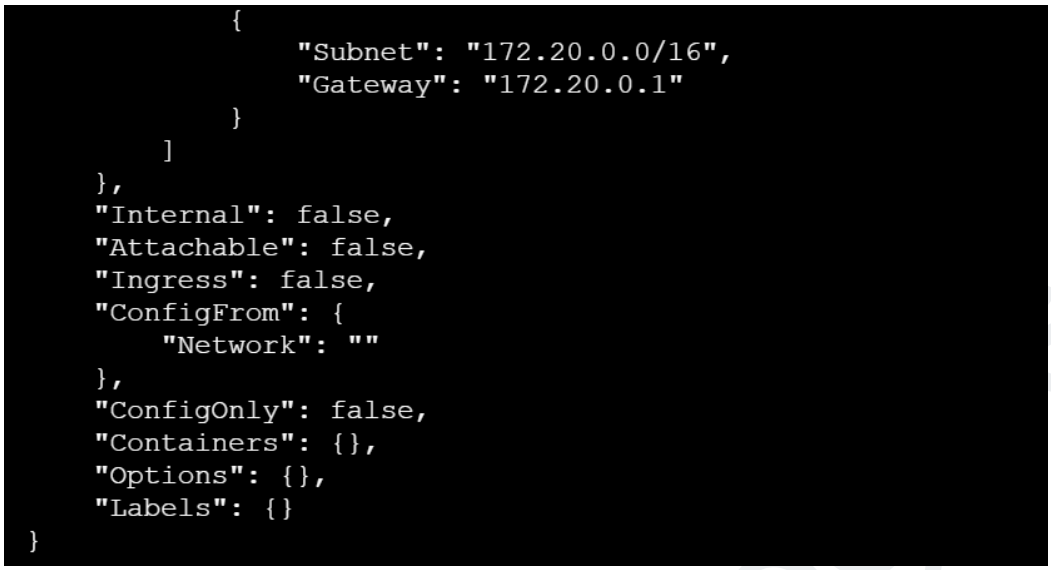
   docker network inspect dev


```
        "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
        "e3bd635db546cd1505b5713dc464031290ac4241ec6f26dfed801f6eca567fb7": {
            "Name": "webpp-app",
            "EndpointID": "25e6ebcb3653011a2659674c2af4552339d52687aad265be2d89fe89b5025107",
            "MacAddress": "02:42:ac:14:00:02",
            "IPv4Address": "172.20.0.2/16",
            "IPv6Address": ""
        }
    },
    "Options": {},
    "Labels": {}
```


5. To remove the network from the container

   docker network disconnect dev webpp-app

Docker

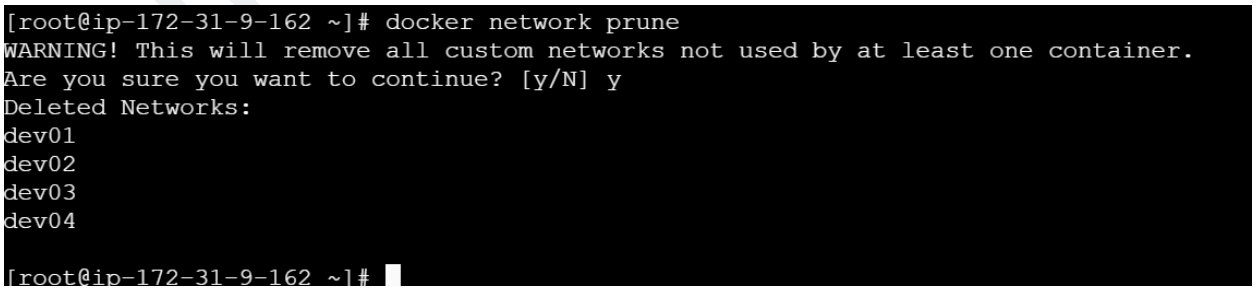


7. To remove the unused networks

    docker network rm dev

8. To remove all unused networks

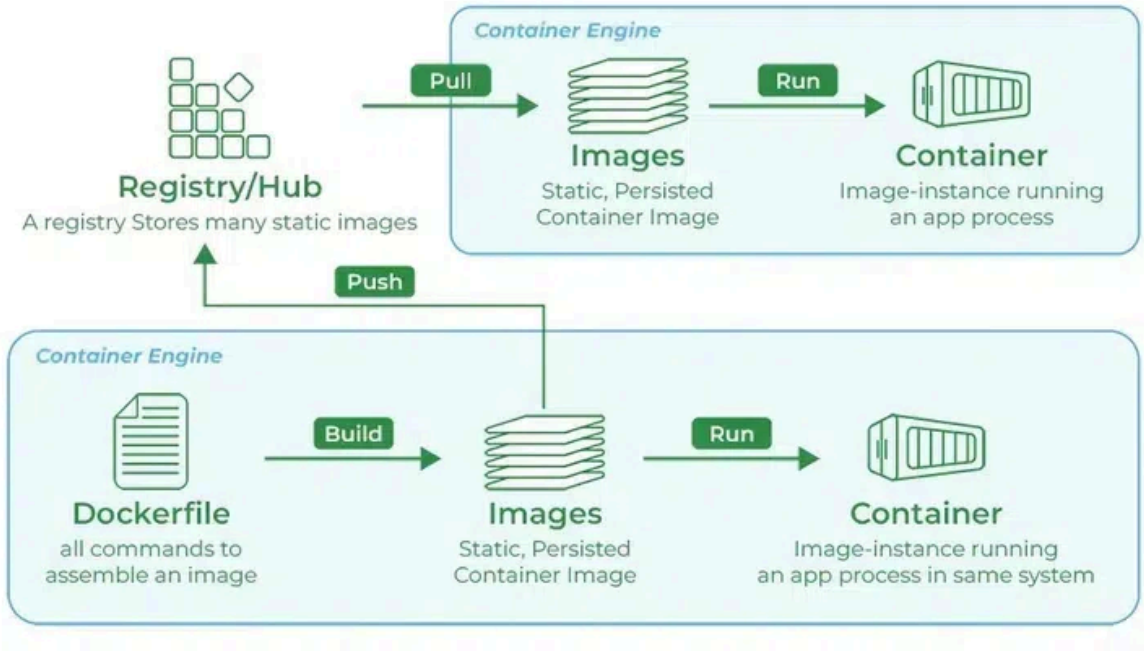
    docker network prune



Docker Registry

Docker

A Docker registry is a system for storing and distributing Docker images with specific names. There may be several versions of the same image, each with its own set of tags. A Docker registry is separated into Docker repositories, each of which holds all image modifications. The registry may be used by Docker users to fetch images locally and to push new images to the registry (given adequate access permissions when applicable). The registry is a server-side application that stores and distributes Docker images. It is stateless and extremely scalable.



What is Docker?

Docker is a container platform that facilitates the developers to package an application with all its dependencies into a single package. It helps in building, shipping and running the containers across the platforms that support docker without any dependency issues.

What is Docker Images

Docker

Docker Image is a light weight executable software template that contains all the dependencies such as software, libraries, runtime. These can be built from the dockerfile which specifies the configuration and steps required to create the image. These Docker images can be maintained in the docker registry as in public or private repositories.

What is Docker Image Registry

Docker Image Registry is a repository for storing and sharing Docker Image. It acts as a centralized location for the developers to upload and download the images. It comes with popular registries including DockerHub where many pre-built images exist.

What is DockerHub

DockerHub is a cloud-based repository that is provided by docker, that allows the users to store and share docker images. It acts as a central hub for the developers in finding the pre-built image for various software applications. It provides both public and private repositories for maintaining the docker images as per the choice to whether it should be publicly exposed or not.

Docker Login

Docker login is a command used in the command line interface that is used for authentication of users with Dockerhub or any other docker registries. On executing the **docker login** command it prompts with asking dockerHub account username and password allowing them to access the private repositories and push or pull the images securely. Try on using the following command for authentication of dockerHub accounts.

Docker

```
docker login
```

What is Docker Distribution

Docker Distribution is an open source project that has features for DockerHub and Docker registries. It provides the backend infrastructure for storing and distributing docker images. It helps in ensuring the efficient and reliable delivery of container images to users worldwide.

What is Authentication of DockerHub

Authentication of Dockerhub involves securely verifying the identity of users who can access the platform. It typically requires users to provide credentials such as username and password or token authentication. On authenticates the users with DockerHub for ensuring only authorized individuals to pull, push or modify the stored images in the repository.

Different Types of Docker Registries

- DockerHub
- Amazon Elastic Container Registry (ECR)
- Google Container Registry (GCR)
- Azure Container Registry (ACR)

Basic commands for Docker registry

Docker

Pull a CentOS image, create a container, install JDK, Tomcat, and Jenkins, then convert the container to an image and push it to Docker Hub, follow these steps:

Step 1: Pull the CentOS Image

docker pull centos:latest (official CentOS)

Step 2: Create and Start a Container from the CentOS Image

docker run -it --name Test-server centos /bin/bash

Step 3: Update the OS

yum update (Incase if you face any issues on the repos, do the below)

1. vi /etc/yum.repos.d/CentOS-Linux-Extras.repo

```
[extras]
name=CentOS-8 - Extras
baseurl=http://vault.centos.org/8.5.2111/extras/x86_64/os/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

2. Update the the below in

vi /etc/yum.repos.d/CentOS-Linux-AppStream.repo

```
[appstream]
name=CentOS-8 - AppStream
baseurl=http://vault.centos.org/8.5.2111/AppStream/x86_64/os/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

3. Update the below in

vi /etc/yum.repos.d/CentOS-Linux-BaseOS.repo

```
[baseos]
name=CentOS-8 - Base
baseurl=http://vault.centos.org/8.5.2111/BaseOS/x86_64/os/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-centosofficial
```

Step 4: Install JDK, Tomcat, and Jenkins Inside the Container

wget https://download.oracle.com/java/17/latest/jdk-17_linux-x64_bin.rpm

yum install wget -y

Install Tomcat

1. wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.91/bin/apache-tomcat-9.0.91.tar.gz
2. tar xzf apache-tomcat-9.0.73.tar.gz
3. mv apache-tomcat-9.0.73 /opt/tomcat
4. /opt/tomcat/bin/startup.sh

Install Jenkins

1. wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat/jenkins.repo rpm --import https://pkg.jenkins.io/redhat/jenkins.io.key
2. yum install -y jenkins
3. Change Jenkins Port and Start Jenkins
    a. sed -i 's/HTTP_PORT=8080/HTTP_PORT=9090/' /etc/sysconfig/jenkins
    b. systemctl start jenkins

Docker

Commit the Container to an Image

1. Exit the container and create an image from it:
   docker commit Test-server hintechnologies-image
2. Tag the Image for Docker Hub
   docker tag hintechnologies-image hintechnologiestrainings/hintechnologies:latest
3. Log in to Docker Hub
   docker login

Push the Image to Docker Hub

1. Push the tagged image to Docker Hub:
   docker push your-dockerhub-username/my-centos-image:latest
   docker push hintechnologies.ittrainings@gmail.com/hintechnologies-image

Docker